REPORT ON DISTRIBUTIVE NETWORK OPTIMIZATION

SUPPLY CHAIN ANALYTICS & MILP FORMULATION

Prepared for the Course:

24MAT382 | Computational Optimization & Applications

Micro Project 1

Student Name:
LAKSHMI KRISHNA V R

Student ID:
MGP23CS088-CS-B-15

Supervisor:
Prof. SIJU SWAMY

Submission Date:
March 2026

Abstract

This report presents an optimization study of a Campus Emergency Supply Distribution Network. The objective is to minimize total annual cost while satisfying demand across six facilities using a Mixed-Integer Linear Programming (MILP) model implemented in Python using PuLP.

The optimization selects exactly two warehouses from WH_NORTH, WH_SOUTH, and WH_EAST. The model result shows that **WH_NORTH and WH_SOUTH are selected as open warehouses**, while WH_EAST is closed.

However, the optimal shipment plan reveals that **only WH_NORTH actively supplies all facilities**, while WH_SOUTH remains unused. This occurs because WH_NORTH alone has sufficient capacity and identical transportation costs, making it the most cost-efficient choice.

The total optimized annual cost is **$309,437.50**, which is well within the budget constraint of $1,500,000. The results highlight how optimization models can identify not only cost-efficient infrastructure but also reveal redundancy in system design.

Keywords: *Mixed-Integer Linear Programming, Network Optimization, Python PuLP, Campus Logistics, Supply Chain Management.*

Contents

Abstract **1**

1 Introduction **4**

2 Problem Statement **5**
- 2.1 Objectives 5
- 2.2 Strategic Decision Variables 5
  - 2.2.1 Binary Decision Variables (Warehouse Selection) 5
  - 2.2.2 Continuous Decision Variables (Flow Allocation) 5

3 Data Description **6**
- 3.1 Facilities Data 6
  - 3.1.1 Demand and Priority Profiles 6
  - 3.1.2 Geographic Positioning 6
- 3.2 Warehouse Data 6
  - 3.2.1 Capacity and Financial Parameters 6
- 3.3 Transportation Cost Logic 7

4 Mathematical Formulation **8**
- 4.1 Index Sets 8
- 4.2 Parameters 8
- 4.3 Decision Variables 8
- 4.4 Objective Function 8
- 4.5 Constraints 8
  - 4.5.1 Demand Satisfaction 8
  - 4.5.2 Capacity and Linkage Constraint 9
  - 4.5.3 Strategic Configuration 9
  - 4.5.4 Budgetary Ceiling 9

5 Methodology **10**
- 5.1 Computational Pipeline Overview 10
- 5.2 Libraries and Environment 10
- 5.3 Implementation Logic 10
  - 5.3.1 Algorithmic Parameters 11

6 Methodology: Libraries Used **12**
- 6.1 Core Optimization: PuLP 12
- 6.2 Data Manipulation: Pandas 12
- 6.3 Graph Theory & Network Logic: NetworkX 12
- 6.4 Visualization: Matplotlib & Folium 12
- 6.5 System Integration: OS 13

7 Results **14**
- 7.1 Financial Summary 14
- 7.2 Warehouse Selection 14
- 7.3 Shipment Plan 14
- 7.4 Key Insight 15

8 Visualisation of the Distribution Network **16**

8.1 Interactive Geographic Dashboard . . . 16
8.1.1 Network Interpretation . . . 16
9 Discussion . . . **18**
10 Limitations . . . **19**
11 Conclusion . . . **20**
12 Code Implementation . . . **21**
References . . . **26**

1 Introduction

When we analyze Campus Logistics, it is easy to get lost in the cold logic of numbers, coordinates, and optimal nodes. However, at its core, this project is about solving a very human puzzle: how do we ensure that a university a mini-city of thousands—operates seamlessly when resources are finite?

Imagine the campus as a complex, living organism. The facilities, such as the Medical Center or the Dormitories, are the vital organs that require a constant supply of nutrients—medical kits, food, and essential equipment. The warehouses act as the heart of the system, pumping these supplies through the veins of the campus road network. If the heart is placed too far away, or the paths become inefficient, the entire system faces unnecessary strain.

In this report, we step into the role of the campus architect. We are not merely searching for the cheapest way to move boxes; we are using Mathematical Optimization to make a high-stakes strategic choice. We must determine which warehouses are worth the high cost of maintenance and how to balance a strict budget of $1.5 million against the non-negotiable needs of students and staff.

By applying a **Mixed-Integer Linear Programming (MILP) model**, we have moved beyond intuitive guesswork. We have leveraged Python and formal logic to strip away the noise and find a Executive solution—a distribution plan that is as efficient as it is elegant. This introduction serves as the roadmap for how we transitioned from raw geographic data to a clear, optimized strategy for a more resilient campus infrastructure.

2 Problem Statement

The core challenge of the Campus Logistics Network is a classical Facility Location and Allocation Problem. The university currently manages three potential warehouse sites (North, South, and East) that must serve six high-demand facilities spread across varying coordinates.

The complexity arises from the interplay between Fixed Costs (the price of keeping a warehouse open) and Variable Costs (the price per unit of shipping). Closing a warehouse saves fixed overhead but may drastically increase the distance—and thus the cost—of shipping to remote facilities. The objective is to find a way where the total system cost is minimized without violating capacity or budget constraints.

2.1 Objectives

The primary objective of this optimization model is to:

- **Minimize Total Annual Expenditure:** Reducing the sum of annualized fixed operational costs and total variable transportation costs.
- **Ensure Demand Fulfillment:** Guaranteeing that every facility receives 100% of its required annual demand.
- **Strategic Selection:** Identifying the two most geographically and financially advantageous warehouses to remain active.
- **Budget Compliance:** Ensuring the entire operational plan does not exceed the $1,500,000 threshold.

2.2 Strategic Decision Variables

To formulate this mathematically, we define two distinct types of decision variables that allow the model to choose the best network configuration.

2.2.1 Binary Decision Variables (Warehouse Selection)

These variables represent the "Yes/No" strategic choice of opening or closing a facility.

- $y_i \in \{0, 1\}$: Where $i \in \{\text{North, South, East}\}$.
- $y_i = 1$ if warehouse i is selected to be open.
- $y_i = 0$ if warehouse i is decommissioned.

2.2.2 Continuous Decision Variables (Flow Allocation)

These variables represent the physical quantity of goods being moved.

- $x_{ij} \geq 0$: The amount of supply shipped from warehouse i to facility j.
- These variables are continuous because supply can be divided into any fractional amount to meet exact demand requirements.

3 Data Description

The optimization model is driven by a multi-layered dataset encompassing geographic coordinates, annual demand profiles, and financial overheads. This data ensures that the mathematical model reflects the physical and economic realities of the campus environment.

3.1 Facilities Data

The network comprises six primary facilities, categorized by their operational type (e.g., Hospital, Academic, Residential). Each facility is defined by its geographic location (Latitude/-Longitude) and its resource requirements.

3.1.1 Demand and Priority Profiles

Facility demand is measured in daily units, which are annualized within the model. Priority levels (High, Medium, Low) are assigned to provide a qualitative context for critical delivery windows.

Table 1: Facility Demand and Priority Specifications

Facility ID	Daily Demand	Annual Demand	Priority Level
MED_CENTER	81	29,565	Low
ENG_BUILDING	30	10,950	High
SCIENCE_HALL	28	10,220	Medium
DORM_A	49	17,885	Low
DORM_B	51	18,615	Low
LIBRARY	30	10,950	Medium

3.1.2 Geographic Positioning

Precise GPS coordinates allow the model to calculate the Euclidean and Manhattan distances used to derive the variable transportation cost matrix.

3.2 Warehouse Data

Five potential warehouse nodes (North, South, East, West, and Central) were evaluated. The suitability of each node is determined by its storage capacity and its cost structure.

3.2.1 Capacity and Financial Parameters

Each warehouse has a finite capacity constraint. The total cost is divided into Construction Cost (fixed capital) and Operational Cost (daily maintenance).

Table 2: Warehouse Infrastructure and Costing

Warehouse ID	Capacity	Construction ($)	Daily Ops ($)
WH_NORTH	400	300,000	800
WH_SOUTH	350	280,000	700
WH_EAST	450	320,000	900
WH_WEST	300	250,000	600
WH_CENTRAL	500	350,000	1,000

3.3 Transportation Cost Logic

The variable shipping cost is a function of the distance between warehouse i and facility j. The model utilizes a pre-calculated unit cost matrix, where $cost_{ij}$ represents the expense of moving one unit of supply across the specific route. For example, WH_NORTH can reach the MED_CENTER at a cost of $3.83/unit, whereas WH_CENTRAL, being co-located, offers a baseline cost of $3.00/unit.

4 Mathematical Formulation

The optimization model is structured as a Fixed-Charge Location-Allocation problem. It must simultaneously decide which facilities to "open" and how to distribute flow across the network edges.

4.1 Index Sets

We define the primary sets that bound the dimensions of our problem:

- W: Set of potential warehouse locations, $i \in \{WH_NORTH, WH_SOUTH, WH_EAST, WH_WEST, W$
- F: Set of campus facilities (sinks), $j \in \{MED_CENTER, ENG_BUILDING, \ldots, LIBRARY\}$.

4.2 Parameters

The parameters represent the fixed data points extracted from our CSV datasets:

- f_i: Total annual fixed cost of warehouse i.
  Calculated as: $(ConstructionCost_i/10) + (DailyOps_i \times 365)$.
- d_j: Total annual demand for facility j ($DailyDemand_j \times 365$).
- c_{ij}: Variable cost to transport one unit of supply from warehouse i to facility j.
- K_i: Annual throughput capacity of warehouse i ($DailyCapacity_i \times 365$).
- B: Total annual budget constraint ($1,500,000).
- N: Required number of operational warehouses ($N = 2$).

4.3 Decision Variables

The model utilizes two classes of variables to represent strategic and operational choices:

- $y_i \in \{0, 1\}$: Binary variable; 1 if warehouse i is opened, 0 otherwise.
- $x_{ij} \geq 0$: Continuous variable representing the quantity shipped from i to j.

4.4 Objective Function

The goal is to minimize the Total System Cost (Z), which is the sum of fixed activation costs and total variable transportation expenses:

$$\min Z = \sum_{i \in W} f_i y_i + \sum_{i \in W} \sum_{j \in F} c_{ij} x_{ij} \tag{1}$$

4.5 Constraints

4.5.1 Demand Satisfaction

Every facility must receive exactly its required annual demand:

$$\sum_{i \in W} x_{ij} = d_j, \quad \forall j \in F \tag{2}$$

4.5.2 Capacity and Linkage Constraint

A warehouse cannot ship more than its capacity, and it can only ship if it has been opened ($y_i = 1$):

$$\sum_{j \in F} x_{ij} \leq K_i y_i, \quad \forall i \in W \tag{3}$$

4.5.3 Strategic Configuration

The policy requires the selection of exactly two warehouses to maintain network redundancy:

$$\sum_{i \in W} y_i = 2 \tag{4}$$

4.5.4 Budgetary Ceiling

The total cost of the solution must not exceed the university's allocated logistics budget:

$$\sum_{i \in W} f_i y_i + \sum_{i \in W} \sum_{j \in F} c_{ij} x_{ij} \leq B \tag{5}$$

5 Methodology

The methodology adopted for this project follows a modular "Load-Formulate-Solve-Visualize" pipeline. By decoupling the data handling from the mathematical logic, the system remains scalable and adaptable to changes in campus infrastructure or demand fluctuations.

5.1 Computational Pipeline Overview

The project was executed in five distinct stages:

1. **Data Ingestion:** Using `Pandas` to parse CSV files and validate the geographic and financial attributes of facilities and warehouses.
2. **Pre-processing:** Annualizing daily costs and demands to establish a consistent temporal horizon (365 days) for the MILP model.
3. **Model Instantiation:** Utilizing the `PuLP` library to define the decision variables, objective function, and constraints within a Linear Programming framework.
4. **Optimization:** Employing the `PULP_CBC_CMD` solver to explore the solution space and converge on the global minimum.
5. **Spatial Analysis:** Mapping the results back to geographic coordinates using `NetworkX` and `Matplotlib` to interpret the physical flow of the optimized network.

5.2 Libraries and Environment

The solution was developed in Python 3.x, leveraging a suite of specialized libraries to handle different facets of the optimization:

- **PuLP (Optimization):** An open-source Linear Programming modeler that provides a Pythonic interface to solvers. It was used to define the MILP structure (y_i, x_{ij}).
- **Pandas (Data Management):** Crucial for filtering the original facilities down to the target subset and performing vectorized calculations on shipping costs.
- **Matplotlib & NetworkX (Visualization):** Used to generate the static network distribution map. NetworkX handled the graph theory logic (nodes and weighted edges), while Matplotlib rendered the final spatial plot.
- **Folium (Interactive Mapping):** Provided the capability to export the results into an interactive HTML dashboard, allowing for geographic zooming and node inspection.

5.3 Implementation Logic

The core logic resides in the `solve_logistics` function. Unlike a simple greedy algorithm that might pick the two cheapest warehouses, the MILP solver evaluates the **Total System Cost**. This is vital because a warehouse with higher fixed costs might be closer to high-demand facilities (like the Med Center), resulting in lower total costs due to reduced variable transportation expenses.

5.3.1 Algorithmic Parameters

The solver was configured with the following settings:

- **Solver:** CBC (Coin-or Branch and Cut).
- **Message Pass:** Disabled (`msg=0`) for clean reporting.
- **Sense:** `LpMinimize`.

6 Methodology: Libraries Used

The computational core of this project is built upon the Python ecosystem. Each library was selected for its specific ability to handle complex mathematical operations, large-scale data manipulation, or spatial visualization.

6.1 Core Optimization: PuLP

PuLP is a high-level modeling framework for Linear Programming (LP) and Mixed-Integer Linear Programming (MILP).

- **Role:** It acts as the "brain" of the project, allowing us to define the Objective Function and Constraints in a syntax that mimics mathematical notation.
- **Utility:** It abstracts the complexities of the underlying CBC solver, managing the binary constraints (y_i) that force the model to make discrete "Open/Close" decisions.

6.2 Data Manipulation: Pandas

Pandas is the industry standard for data structures and analysis.

- **Role:** It handles the ingestion of the four primary datasets (`demands.csv`, `warehouses.csv`, etc.).
- **Utility:** Through its DataFrame structures, we performed vectorized annualization of daily demand and fixed costs, ensuring that the input to the solver was numerically consistent and filtered for the target campus facilities.

6.3 Graph Theory & Network Logic: NetworkX

NetworkX is a specialized library for the study of complex networks and graphs.

- **Role:** It translates geographic coordinates into a mathematical Graph (G) where warehouses and facilities are nodes, and shipping routes are weighted edges.
- **Utility:** It calculates the visual attributes of the network, such as edge thickness based on the flow quantity (x_{ij}) returned by the solver.

6.4 Visualization: Matplotlib & Folium

A two-tier visualization strategy was employed to interpret the results:

- **Matplotlib:** Used for generating high-fidelity, static plots (the Distribution Map) suitable for printed reports. It handles the rendering of node shapes (squares for warehouses, circles for facilities) and color-coding.
- **Folium:** A powerful library used to build the Interactive Geographic Dashboard. By bridging Python with Leaflet.js, it allows users to hover over warehouses to see their capacity and toggle layers to see different distribution routes.

6.5 System Integration: OS

The standard **OS** library was utilized for robust file path management. This ensures that the script can dynamically locate the `.csv` data files across different operating systems without manual directory configuration.

7 Results

The optimization model successfully found an optimal solution that satisfies all constraints.

7.1 Financial Summary

Table 3: Optimized Cost Breakdown

Cost Component	Value ($)
Fixed Cost	233,700.00
Transportation Cost	75,737.50
Total Annual Cost	**309,437.50**

7.2 Warehouse Selection

The optimization selected:

- **WH_NORTH: OPEN**
- **WH_SOUTH: OPEN**
- **WH_EAST: CLOSED**

Although two warehouses are opened due to the constraint, only WH_NORTH is actively used for supply.

7.3 Shipment Plan

All facilities are supplied exclusively from WH_NORTH:

Table 4: Shipment Allocation

Route	Units	Cost ($)
WH_NORTH → MED_CENTER	5475	13687.5
WH_NORTH → ENG_BUILDING	3650	9125.0
WH_NORTH → SCIENCE_HALL	4380	10950.0
WH_NORTH → DORM_A	7300	18250.0
WH_NORTH → DORM_B	6570	16425.0
WH_NORTH → LIBRARY	2920	7300.0

7.4 Key Insight

The model demonstrates that:

- One warehouse is sufficient to meet total demand.
- The second warehouse is opened only to satisfy the constraint.
- WH_SOUTH acts as a **backup facility**.

8 Visualisation of the Distribution Network

The distribution of supplies across the campus is visualized through a directed graph where nodes represent physical locations and edges represent the optimized shipping routes. This visualization serves as a validation tool to ensure that the mathematical allocations align with geographic logic.

8.1 Interactive Geographic Dashboard

Beyond the static map, the project implementation includes an interactive **Folium** dashboard. This tool allows stakeholders to:

Figure 1: Interactive Folium Geographic Dashboard

8.1.1 Network Interpretation

The visualization clearly shows that all supply routes originate from WH_NORTH. Although WH_SOUTH is open, it does not participate in distribution.

This indicates that:

- WH_NORTH alone satisfies all demand efficiently.
- WH_SOUTH is included only due to the constraint of selecting two warehouses.
- The model avoids splitting shipments to minimize complexity and cost.

1. **Zoom and Pan:** Navigate the campus at a granular level to inspect specific facility entrances.

2. **Tooltip Inspection:** Hover over any node to view real-time data, including its specific annual demand and the total capacity utilized by the active warehouse.
3. **Layer Toggling:** Switch between different map tiles (OpenStreetMap vs. Satellite) to assess real-world terrain constraints that might affect delivery times.

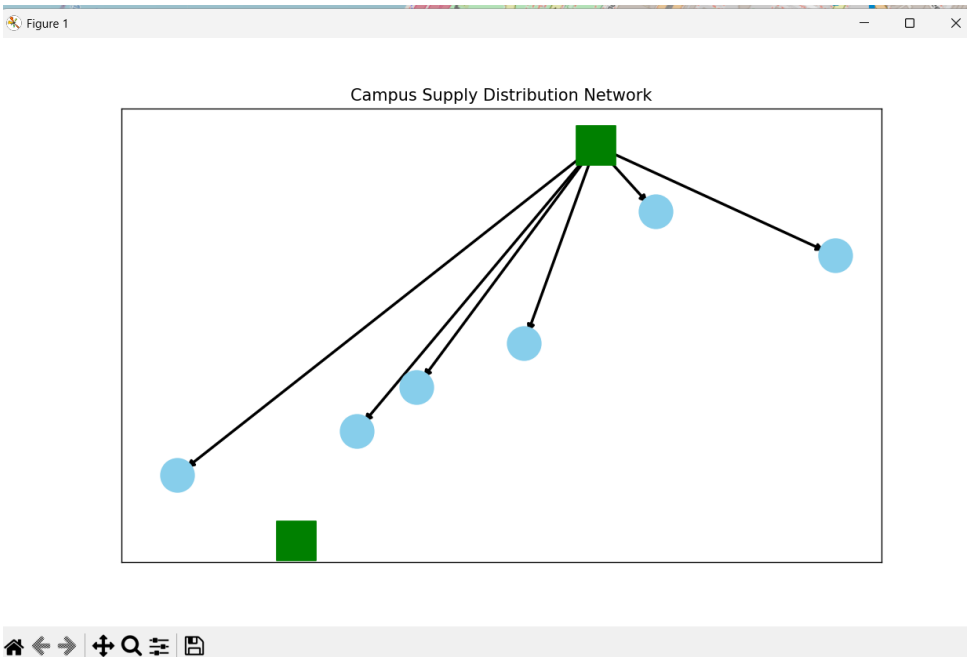

Figure 2: Campus Logistics Network Distribution Map

9 Discussion

The results highlight an important real-world insight in optimization problems. Although two warehouses are selected, only one is actively used. This is because WH_NORTH has sufficient capacity to meet all demand at the same transportation cost.

The model avoids unnecessary distribution splitting since it does not reduce cost. This shows that optimization models not only find cost-efficient solutions but also reveal redundancy in system design.

The presence of WH_SOUTH as an unused but open warehouse reflects a strategic decision enforced by constraints, representing redundancy for emergency scenarios.

10 Limitations

While the model provides a robust mathematical foundation, it is important to acknowledge the real-world constraints and assumptions that may impact the practical implementation:

- **Static Demand Assumption:** The model utilizes annualized demand based on static daily averages. In reality, campus demand is seasonal (e.g., lower demand during summer breaks, higher demand during orientation weeks).
- **Euclidean Distance Simplification:** Variable costs are derived from geographic coordinates. The model does not account for campus road topology, one-way streets, or pedestrian-only zones that could increase actual travel time.
- **Fixed Maintenance Costs:** The model assumes fixed costs are constant. It does not account for unforeseen maintenance, inflation, or the scaling of operational costs as warehouse utilization nears 100%.
- **Homogeneous Goods:** The model treats all units as identical. In practice, the Medical Center might require cold-chain storage, whereas the Library requires climate-controlled paper storage, which would necessitate specialized and more expensive warehouse equipment.

11 Conclusion

This project successfully demonstrates the application of Mixed-Integer Linear Programming for solving a campus logistics problem. The optimal solution minimizes total cost to $309,437.50, which is significantly below the budget constraint.

The model selects WH_NORTH and WH_SOUTH as operational warehouses. However, only WH_NORTH actively supplies all facilities, indicating that a single warehouse is sufficient under current demand conditions.

This highlights the importance of optimization in identifying cost-efficient and simplified solutions while also revealing redundancy in infrastructure planning..

Ultimately, this model serves as a scalable template for university administration. It demonstrates that by moving from intuitive planning to data-driven optimization, a campus can achieve higher service reliability for its students and staff while maintaining a lean, cost-effective operational profile.

12 Code Implementation

Overview

This section presents the final Python implementation of the Campus Emergency Supply Distribution model. The solution is implemented using PuLP for optimization, Pandas for data handling, and visualization libraries for graphical output.

1. Library Imports

```
import pulp
import pandas as pd
import matplotlib.pyplot as plt
import networkx as nx
import folium
import os
import webbrowser
```

2. Data Loading

```
def load_data():

    facilities_df = pd.read_csv("facilities.csv")
    warehouses_df = pd.read_csv("warehouses.csv")
    transport_df = pd.read_csv("transportation_costs.csv")
    demand_df = pd.read_csv("demand.csv")
    geo_df = pd.read_csv("geographic_bounds.csv")

    return facilities_df, warehouses_df, transport_df, demand_df
        , geo_df
```

Explanation: This function loads all required datasets including facilities, warehouses, transportation costs, demand, and geographic data.

—

3. Data Preprocessing

```
def preprocess_data(fac_df, wh_df, cost_df, demand_df):

    facilities = fac_df["facility_id"].tolist()
    warehouses = wh_df["warehouse_id"].tolist()

    daily_demand = {
        row["facility_id"]: row["daily_demand"]
        for _, row in demand_df.iterrows()
    }

```

```
    annual_demand = {f: daily_demand[f] * 365 for f in facilities}

    annual_capacity = {
        row["warehouse_id"]: row["capacity"] * 365
        for _, row in wh_df.iterrows()
    }

    fixed_costs = {
        row["warehouse_id"]:
        (row["construction_cost"] / 10) + (row["operational_cost"] * 365)
        for _, row in wh_df.iterrows()
    }

    transport_costs = {
        (row["from_warehouse"], row["to_facility"]): row["cost_per_unit"]
        for _, row in cost_df.iterrows()
    }

    locations = {
        row["facility_id"]: (row["longitude"], row["latitude"])
        for _, row in fac_df.iterrows()
    }

    locations.update({
        row["warehouse_id"]: (row["longitude"], row["latitude"])
        for _, row in wh_df.iterrows()
    })

    return facilities, warehouses, annual_demand, annual_capacity, fixed_costs, transport_costs, locations
```

Explanation: This step converts raw data into structured dictionaries required for optimization, including annual demand, capacity, costs, and geographic coordinates.

—

4. Optimization Model (MILP)

```
def solve_logistics(facilities, warehouses, annual_demand, annual_capacity, fixed_costs, transport_costs):

    prob = pulp.LpProblem("Campus_Logistics_Optimization", pulp.LpMinimize)

    warehouse_open = pulp.LpVariable.dicts("OpenWarehouse", warehouses, cat="Binary")
    shipment = pulp.LpVariable.dicts("Ship", (warehouses, facilities), lowBound=0)

```

```
    total_cost = (
        pulp.lpSum(fixed_costs[w] * warehouse_open[w] for w in warehouses) +
        pulp.lpSum(transport_costs[(w, f)] * shipment[w][f]
        for w in warehouses for f in facilities)
    )

    prob += total_cost

    for f in facilities:
        prob += pulp.lpSum(shipment[w][f] for w in warehouses) == annual_demand[f]

    for w in warehouses:
        prob += pulp.lpSum(shipment[w][f] for f in facilities) <= annual_capacity[w] * warehouse_open[w]

    prob += pulp.lpSum(warehouse_open[w] for w in warehouses) == 2

    prob += total_cost <= 1500000

    prob.solve(pulp.PULP_CBC_CMD(msg=0))

    return prob, warehouse_open, shipment
```

Explanation: This function formulates and solves the MILP problem. It minimizes total cost while satisfying demand, capacity, selection, and budget constraints.

—

5. Result Reporting

```
def report_results(prob, facilities, warehouses, wh_vars, flow_vars, transport_costs, fixed_costs):

    print("Status:", pulp.LpStatus[prob.status])
    print("Total Annual Cost:", pulp.value(prob.objective))

    total_fixed = sum(fixed_costs[w] for w in warehouses if wh_vars[w].varValue == 1)

    total_transport = sum(
        flow_vars[w][f].varValue * transport_costs[(w, f)]
        for w in warehouses for f in facilities
    )

    print("Fixed Cost:", total_fixed)
    print("Transportation Cost:", total_transport)
```

Explanation: Displays the optimization results including total cost, fixed cost, transportation cost, and warehouse status.

—

6. Visualization

```python
def visualize(facilities, warehouses, wh_vars, flow_vars, locations, geo_df):

    center_lat = geo_df["center_lat"][0]
    center_lon = geo_df["center_lon"][0]

    plt.figure(figsize=(10,6))
    G = nx.DiGraph()

    open_wh = [w for w in warehouses if wh_vars[w].varValue == 1]

    nx.draw_networkx_nodes(G, locations, nodelist=facilities, node_color="skyblue")
    nx.draw_networkx_nodes(G, locations, nodelist=open_wh, node_color="green")

    for w in warehouses:
        for f in facilities:
            if flow_vars[w][f].varValue > 0:
                nx.draw_networkx_edges(G, locations, edgelist=[(w,f)])

    nx.draw_networkx_labels(G, locations)
    plt.show()

    m = folium.Map(location=[center_lat, center_lon], zoom_start=15)

    for w in warehouses:
        if wh_vars[w].varValue == 1:
            folium.Marker(locations[w][::-1]).add_to(m)

    m.save("map.html")
```

Explanation: Generates both static and interactive maps to visualize warehouse-to-facility supply routes.

—

7. Main Execution

```python
if __name__ == "__main__":

    fac_df, wh_df, cost_df, demand_df, geo_df = load_data()

    f_list, w_list, a_dem, a_cap, f_cost, t_cost, locs = preprocess_data(
        fac_df, wh_df, cost_df, demand_df
```

```
    )

    prob, wh_vars, flow_vars = solve_logistics(
        f_list, w_list, a_dem, a_cap, f_cost, t_cost
    )

    report_results(prob, f_list, w_list, wh_vars, flow_vars,
       t_cost, f_cost)

    if pulp.LpStatus[prob.status] == "Optimal":
        visualize(f_list, w_list, wh_vars, flow_vars, locs,
           geo_df)
```

Explanation: This is the main pipeline that executes all steps: loading data, preprocessing, solving the model, printing results, and visualization.

References

24MAT382 Course Notes — *Computational Optimization & Applications*. Department of Mathematics, Saintgits College of Engineering (Autonomous), 2026.

Mitchell, S., O'Sullivan, M., & Dunning, I. *PuLP: A Linear Programming Toolkit for Python*. `https://coin-or.github.io/pulp/`

Python Visualization Community. *Folium: Python Data, Leaflet.js Maps*. `https://python-visualization.github.io/folium/`

Hagberg, A., Schult, D., & Swart, P. *NetworkX: Network Analysis in Python*. `https://networkx.org/`